

Data structures and algorithms

Assignment 5

Vinnyk Yehor

The assignment requires implementation of different data structures and compare their efficiency in execution. For this task I have chosen AVL tree, Splay tree, hash table with chaining and double hashing.

1. AVL tree

AVL tree is type of binary tree, where all the keys are sorted and after each mutable operation we count balance factor $BF = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$, and if balance factor is out of the interval $-1 \leq BF \leq 1$ we should perform rotations in order to balance the tree. Every node has height value. We start balancing tree from the bottom to top, to be convinced that nodes on all levels are balanced. The complexity of all operations for such tree is $O(\log n)$ where n is number of nodes.

Structure of AVL tree

```
struct avlTreeNode
{
    int key;
    int height;
    struct avlTreeNode *leftNode;
    struct avlTreeNode *rightNode;
    struct avlTreeNode *parentNode;
};
```

Int key – key of the node, which is used for searching for a node.

Int height – height property of node

Struct avlTreeNode *leftNode – pointer to left node, which has key less than current node.

Struct avlTreeNode *rightNode – pointer to right node, which has key greater than current node.

Struct avlTreeNode *parentNode – pointer to closer ancestor, the parent of the current node.

Function and their role

- **avlCreateNode()** – creating new AVL node that will store passed key, height, pointers to parent structure and children structures.
- **getHeight()** – checking whether the passed node is null, if not, return its height
- **rotateRight()** – if balance factor is more than 1, we perform rotation to the right (since left subtree is bigger) by changing the pointers to structures and updating heights of rotated nodes.
- **rotateLeft()** – the same, but rotating is performed to the left, when right subtree is bigger.
- **rebalance()** – updating the height of the root of the subtree we are checking, compute balance factor and depending on balance factors of subtrees perform corresponding rotations.
- **avlInsertNode()** – function for inserting nodes, we check through the tree if node with given key exists, if yes - we return root, if no – we create a new node and make the previous one point to it. Then we go up through the tree, calculation balance factor and perform rotations for every subtree if its needed. If there wasn't parent, it means than we have rebalanced the root and return subtree, If there was parent we check If the subtree we balanced was right or left child and put pointers to corresponding structures. Finally, return the new balanced tree.
- **avlDeleteNode()** – in delete function we go through tree and check if the node with given key exists, if no – we return root and delete nothing, if yes – we find this node. If the node has 2 children, we go to first right child and then to last left child, copy its value to node that we are going to delete and now node that will be deleted is the very left node of right subtree, we delete it. In case if it has at least one child node or none, we connect parent node of node that is going to be deleted with child node or null, and delete needed node. Final step is to do balancing. Return root.
- **avlFindNode()** – this function is the same as delete, but without deletion part, we just travelling the tree and looking for node with given key.

There were also implemented function for inorder, preorder, postorder (recursions), but they are much less efficient for AVL tree, than binary search. The functions are implemented iteratively, since recursion is memory

consuming, especially when number of nodes is huge. `avlFreeTree()` frees the tree.

2. Splay tree

Splay tree is the tree where after each operation we move specific node to the root of the tree. Speed $O(\log n)$.

Structure of Splay tree

```
struct splayTreeNode
{
    int key;
    struct splayTreeNode *leftNode;
    struct splayTreeNode *rightNode;
    struct splayTreeNode *parentNode;
};
```

Int key – key of the node, which is used for searching for a node.

Struct `avlTreeNode *leftNode` – pointer to left node, which has key less than current node.

Struct `avlTreeNode *rightNode` – pointer to right node, which has key greater than current node.

Struct `avlTreeNode *parentNode` – pointer to closer ancestor, the parent of the current node.

Functions and their role

- **`createSplayNode()`** - creating new Splay node that will store passed key, pointers to parent structure and children structures.
- **`splayRotateRight()`** - we perform rotation to the right by changing the pointers to structures.
- **`splayRotateLeft()`** - the same, but rotating is performed to the left.
- **`toSplay()`** – in this function we perform rotations until the inserted, deleted or visited node is at the root. Updating pointers correctly on each step.

- **splayInsertNode()** – the same as for AVL tree, but splaying at the end instead of balancing.
- **splayDeleteNode()** – the same search for node with specific key, splaying it to the root, deleting it and return right subtree if left subtree of deleted node is NULL, or looking for the biggest node in left subtree, splaying it to the root of left subtree and make it point to right subtree root, this will make this node new root of the tree. Return new root.
- **splayFindNode()** – the same as delete function, but without delete operation, just splaying searching node to the root.

There are also implementations for inorder, preorder, postorder searches as in AVL tree, but for Splay tree they are more efficient, their comparison with binary search will be described lower. **splayFreeTree()** – function for freeing memory of tree.

3. Hash table with chaining

Hash table is a table that stores data by specific hash index which is calculated by hash function. Function has key and unique value. Chaining method helps resolve collisions in table by creating linked lists if several structures should have the same hash index. Speed $O(1)$.

Structures of hashing table with chaining

```
struct chainingHashNode
{
    char key[30];
    int value;
    struct chainingHashNode *nextNode;
};

struct chainingHashTable
{
    struct chainingHashNode **table;
    int size;
    int capacity;
};
```

Char key[30] – key of Node

Int value – value

Struct chainingHashNode *nextNode – pointer to node, will be used for creating linked lists.

Struct chainingHashNode ** table – array of pointers, for implementing linked lists.

Int size – size of the table

Int capacity – capacity of table.

Functions and their role

- **chainingCreateTable()** – function for creating table structure, assigning values of capacity, size and allocating memory for array of pointers(table).
- **hashFunction()** - function for calculation hash index using passed key.
- **Resize()** – resizing table, doubling the capacity, allocating memory for array of pointers(table) due to new capacity, adding to new created table elements that were stored before.
- **chainingCreateNode()** – creating new node.
- **chainingInsertNode** – computing load factor, if it is more than 0,75, we resize the table, we do, since It helps as to make hash table more efficient and avoid additional collisions. Then we calculate hash index for specific key, creating new node and storing it by this index, if there was already linked list stored by that index, we just add new element to the head of this linked list, otherwise we just store that structure in our table and table points to it. In the end increase size of table.
- **chainingDeleteNode()** – we get hash index using passed key and searching for structure with that key through linked list stored by that hash index. When we find element we free it and decreasing size of table by one.
- **chainingFindNode()** – the same as deleting function, but without deleting part, just searching for a specific node.
- **chainingFreeTable()** – free memory of hash table.

4. Hash table with double hashing

Hash table with double hashing resolves collisions by using one more hash function that determines the length of “step” that program will do to find the nearest free space for new structure node.

```

struct doubleHashNode
{
    char key[30];
    int value;
    int status;
};

struct doubleHashTable
{
    struct doubleHashNode *table;
    int size;
    int capacity;
};

```

Everything is the same as in hash table with chaining, but table is an array of structures, but not an array of pointers.

Functions and their role

- **doubleCreateTable()** – creates hash table as hash table described above does, but now table is an array of structures and allocate with calloc in order to store condition of every node in table (status), 0 – free(empty), 1 – contains structure, 2 – structure was deleted.
- **doubleHashFunction()** – exactly the same hash function as in hash table with chaining
- **doubleHashFunction1()** – computes so called “step”, that allows us to find nearest free cell in table.
- **doubleResize()** – resize function, that follows the same principle as hash table with chaining, but with this own features.
- **doubleInsertNode()** – checking if table needs to be resized, computes hash index and second hash index. Iterating through table and searching for empty or freed cell, if it finds such cell, it fills data to structure stored in that cell, if structure in current cell that is checking exists and its key the same as the key of structure that should be inserted, it means that the same structure is going to be added, so we just change the value of current structure to that on that was given, increase size by 1.
- **doubleDeleteNode()** – we move through the table, if we find first empty cell, it means that no sense in further searching, because the rest of cells in that path are also empty, we delete nothing, but If cell is filled with structure and

fields coincide, we delete that structure(give it status 2) and decrease the size by 1

- **doubleFindNode()** – we looking for structure the same as we do in delete function, but without setting the status, if node is found we stop the function.
- **doubleFreeTable()** – free hash table.

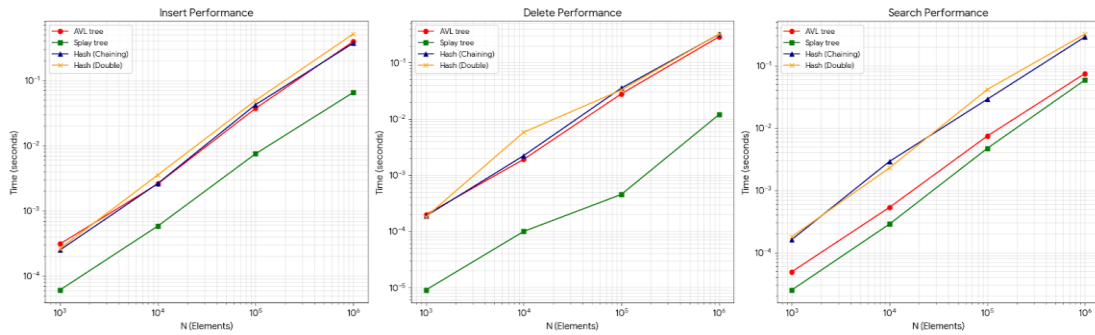
Testing

The test are executed in separate file. Time received in testing is in seconds.

1. Comparing of all three algorithms

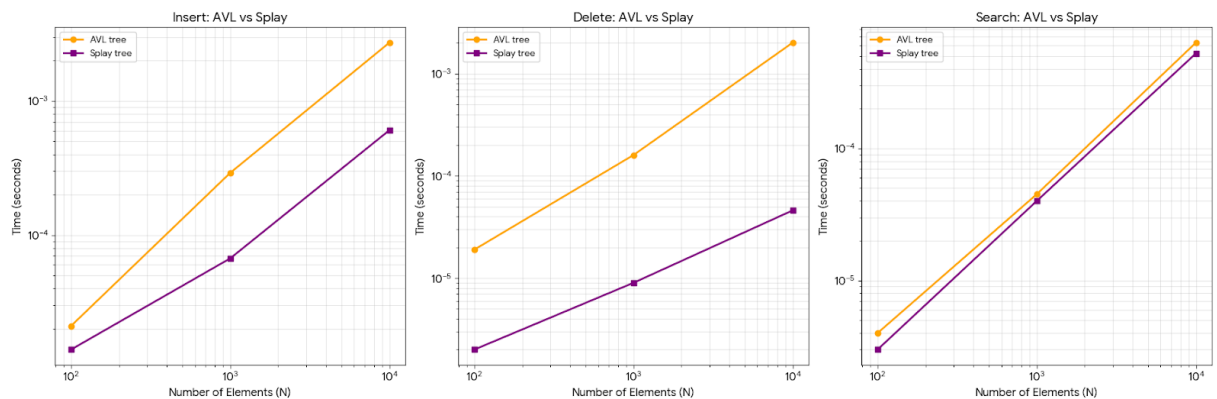
Operation	Structure	1000	10000	100000	1000000
insert	AVL tree	0.000310 s	0.002581 s	0.036864 s	0.391722 s
	Splay tree	0.000061 s	0.000581 s	0.007438 s	0.064864 s
	Hash table with chaining	0.000250 s	0.002611 s	0.042017 s	0.369231 s
	Hash table with double hashing	0.000263 s	0.003526 s	0.048833 s	0.511038 s
delete	AVL tree	0.000197 s	0.001897 s	0.027855 s	0.289316 s
	Splay tree	0.000009 s	0.000099 s	0.000452 s	0.011927 s
	Hash table with chaining	0.000186 s	0.002211 s	0.035224 s	0.320670 s
	Hash table with double hashing	0.000180 s	0.005752 s	0.032602 s	0.323434 s
search	AVL tree	0.000049 s	0.000530 s	0.007383 s	0.074640 s
	Splay tree	0.000025 s	0.000286 s	0.004686 s	0.058763 s
	Hash table with chaining	0.000161 s	0.002895 s	0.028802 s	0.288554 s
	Hash table with double hashing	0.000179 s	0.002285 s	0.041256 s	0.321002 s

Note: tests for 1 million elements for hash tables might not work, because of weak equipment. I have experienced it and the solution is to run only tests for hash tables and only 1 million nodes.



Comparison of trees

Operation	Algorithm	100	1000	10000
insert	AVL tree	0.000021 s	0.000291 s	0.002722 s
	Splay tree	0.000014 s	0.000067 s	0.000605 s
delete	AVL tree	0.000019 s	0.000160 s	0.002012 s
	Splay tree	0.000002 s	0.000009 s	0.000046 s
search	AVL tree	0.000004 s	0.000045 s	0.000632
	Splay tree	0.000003 s	0.000040 s	0.000524 s



The final step of comparison is different search methods in trees: Inorder, Postorder and Preorder.

For AVL tree these methods are not efficient at all: for 10000 – 2.6236(inorder), 2.6001(preorder), 2.6153(postorder), for 100000 – 26.6593(inorder), 26.4716(preorder), 26.5713(postorder). This is because these methods neglect the fact that the tree is sorted and operations can be done more efficiently.

For splay tree it is for 100000 elements – 0.0011(inorder), 0.00126(preorder), 0.001243(postorder). As we can see, it is almost 4 times more efficient for 100000 elements, which makes these methods really good for splay trees, but the best is inorder.